

Smart Pizza Shop Management System

Project Proposal — Web-Based, Multi-Tenant Pizza Shop Management Platform

1. Project Overview

The Smart Pizza Shop Management System is a commercial web application designed to manage the daily operations of pizza shops and their branches. The system combines order management, inventory, purchasing, kitchen operations, customer management, financial tracking, analytics, and intelligent recommendations in one platform. The architecture is designed from the beginning to support multiple independent businesses (multi-tenancy), making the system suitable for future commercialization as a SaaS product.

2. Problem Statement

Many small and medium-sized food businesses manage orders, inventory, purchasing, expenses, and customer information using disconnected tools or simple point-of-sale solutions. These approaches often provide limited insight into stock consumption, waste, supplier performance, demand patterns, and kitchen workload. This project aims to provide a centralized system that not only records business operations but also turns operational data into useful recommendations.

3. Main Objectives

- Build a professional web-based management platform for pizza shops.
- Provide a scalable backend using Python and FastAPI.
- Use PostgreSQL as the primary relational database.
- Support multiple independent businesses and multiple branches through a multi-tenant architecture.
- Improve inventory, purchasing, kitchen, order, and financial management.
- Provide analytics and intelligent recommendations based on historical and operational data.
- Design the system so future commercial features such as subscriptions and billing can be added without major architectural changes.

4. Core Functional Modules

Module	Main Capabilities
Product & Menu	Products, categories, variants, single/two-size products, add-ons/options, availability.
Recipe Management	Recipes connected to ProductVariant, ingredient quantities, standardized units.
Order Management	DINE_IN, TAKEAWAY, DELIVERY, order items, discounts, payments, status tracking.
Kitchen Management	Kitchen Order View, PREPARING status, kitchen capacity and workload tracking.
Customer Management	Customer profiles, addresses, order history, recommendations.
Inventory	Stock levels, batches, expiry dates, inventory transactions, waste, FEFO support.
Purchasing	Suppliers, purchases, purchase items, ingredient costs, supplier performance.
Financial Management	Purchase costs, business expenses, sales, financial summaries and reports.
Analytics	Sales, consumption trends, supplier analysis, waste analysis and business summaries.
Discounts	Discount codes, usage limits, validity periods and customer usage tracking.

5. Smart & Innovative Features

Smart Management Dashboard — Analyzes sales and operations, predicts busy hours, identifies risks, and provides inventory/purchasing recommendations.

Smart Inventory & Restocking — Estimates required purchase quantities using current stock, consumption trends, safety stock, supplier lead time and expected demand.

Demand Prediction — Uses historical sales patterns, day/time and operational data to estimate future demand.

Customer Recommendations — Suggests complementary products and frequently bought together combinations based on customer and aggregate order behavior.

Dynamic Preparation / Delivery Prediction — Estimates preparation and delivery time using current orders, kitchen capacity, historical preparation times and order complexity.

Expiry & Waste Intelligence — Tracks batches and expiry dates and helps prioritize stock usage and identify waste patterns.

Supplier Analysis — Compares supplier prices, purchase trends and performance over time.

6. Multi-Tenant & Multi-Branch Architecture

The database will be designed for multiple independent businesses from the beginning. Each business is represented by a Tenant and may contain multiple Branches. Tenant-scoped data will be isolated from other businesses. PostgreSQL Row-Level Security (RLS) will be implemented where appropriate as an additional database-level security layer. Branch-level data such as inventory, orders, expenses and kitchen operations will be associated with the relevant branch.

7. Technology Stack

Layer	Technology / Approach
Backend	Python + FastAPI
Database	PostgreSQL
ORM / Data Access	SQLAlchemy
Database Migrations	Alembic
Validation	Pydantic
API Standard	REST API + OpenAPI
API Documentation	FastAPI Swagger UI (/docs) and ReDoc (/redoc)
Authentication	Secure authentication and authorization architecture
Testing	Pytest and automated API/service tests
Frontend	Separate web frontend repository; technology to be selected and learned during implementation
Deployment	Containerized/deployable architecture, with Docker considered for development and deployment

8. Backend Architecture

The backend will follow a layered architecture separating API routes, validation schemas, business services, repositories, database models, and core configuration. This separation will make the system easier to test, maintain and extend.

Typical request flow: Frontend → FastAPI Route → Pydantic Schema → Service Layer → Repository/Data Access → PostgreSQL.

9. Documentation Standards

- Meaningful docstrings will be written for public classes, functions and important modules.
- API routes will have clear summaries, descriptions, parameters, request models and response models.
- FastAPI-generated OpenAPI documentation will be available through /docs and /redoc.
- A separate API documentation section will describe endpoints, authentication, request/response examples and possible errors.
- Architecture and database documentation will be maintained alongside the project.

10. Security & Data Integrity

- Tenant data isolation using tenant-aware application logic and PostgreSQL RLS.
- Role-based access control for different staff roles.
- Secure password hashing and authentication.
- Database constraints, foreign keys, validation and transactions.
- Audit logging for important business and administrative actions.
- Environment variables for secrets and configuration.

11. Future Commercial Expansion

The architecture will be SaaS-ready. Subscription plans, billing, invoices and subscription payments are intentionally not part of the initial implementation, but the Tenant-based architecture will allow these features to be introduced later without redesigning the entire system.

12. Project Development Plan

1. Finalize database entities, relationships and constraints.
2. Implement PostgreSQL database schema and migrations.
3. Implement FastAPI backend structure and database integration.
4. Design and implement REST API routes.
5. Implement authentication, authorization, multi-tenancy and RLS.
6. Implement products, recipes, orders, inventory, purchasing and financial modules.
7. Implement analytics and intelligent recommendation features.
8. Write tests and improve error handling, logging and security.
9. Maintain API, architecture and code documentation.
10. Develop the separate frontend repository and connect it to the backend APIs.

13. Expected Outcome

The expected result is a scalable, commercial-oriented web application that can be used to manage a real pizza shop and can later be offered to multiple businesses. The system will combine conventional business management with data-driven and intelligent features to improve operational efficiency, inventory decisions, customer experience and management visibility.